Toward Developing High-Performance RISC-V Processors Using Agile Methodology

Yinan Xu and Zihao Yu, *State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, 100190, China*

Dan Tang, *Beijing Institute of Open Source Chip, Beijing, 100080, China*

Ye Cai, *Shenzhen University, Shenzhen, 518060, China*

Dandan Huan, *Beijing VCore Technology, Beijing, 100190, China*

Wei He, *Peng Cheng Laboratory, Shenzhen, 518060, China*

Ninghui Sun and Yungang Bao, *State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, 100190, China*

Agile chip design methodology has shown promise for sustaining the scaling of computing performance more efficiently. However, the practical application of this methodology has been limited by major obstacles. This article presents MinJie, an open source platform supporting agile hardware development flow. We demonstrate the usage and effectiveness of MinJie by building two generations of XiangShan, an open source RISC-V processor with industry-competitive performance. This article highlights the potential impact of MinJie and XiangShan in advancing the field of agile processor design and development as well as the potential for open source collaboration to democratize the field and drive innovation forward.

In the data-driven world, chip architects need to carefully balance conventional design metrics, such as performance, power, and area (PPA), against other metrics, such as time to market and limited budgets, when designing high-performance processors. This requires novel methods and tool flows that oversee and facilitate the entire process.

In recent years, the agile chip design methodology has attracted attention from both academia and industry. These approaches have the potential to reduce the significant engineering costs and long design cycles for chip development while maintaining reasonable PPA specifications. There is an increasing amount of research[1,2,3] demonstrating the effectiveness of agile methodologies in designing CPU cores, systems on chip, and hardware accelerators.

Despite its benefits, the use of agile methodology still needs to be expanded, especially for practical modern designs. To further investigate this issue, we talked to chip architects, research scientists, and lead engineers from 12 hyperscalers and major chip companies across multiple sectors, including data centers, smartphones, communications, and automobiles. According to this survey, we summarize two common concerns regarding the agile methodology:

- Agile approaches are not quite ready for *complicated* processors. Although a number of chips have been built using agile approaches, most of them are research prototypes and are relatively small or have less complicated designs. It is still unclear if similar approaches can be directly applied to large-scale designs, such as modern high-performance processors.
- The verification process is still *less agile*. Despite modern methodologies widely adopted to accelerate the verification process, it remains a major bottleneck in the entire chip development workflow, especially for large designs. Existing frameworks for agile development focus more on rapid prototyping than on verification. Therefore, more

0272-1732 © 2023 IEEE
Digital Object Identifier 10.1109/MM.2023.3273562
Date of publication 9 May 2023; date of current version 29 June 2023.

Authorized licensed use limited to: ShanghaiTech University. Downloaded on November 10,2026 at 03:16:58 UTC from IEEE Xplore. Restrictions apply.
Published by the IEEE Computer Society

tools are required to support the entire agile workflow.

In this article, to address both concerns, we present MinJie, a platform that integrates a rich set of agile development tools for logic design, functional verification, performance modeling, and debugging, to further improve the development efficiency of state-of-the-art processor designs.

To demonstrate the usage and capability of this platform, we develop XiangShan, a high-performance, superscalar, out-of-order RISC-V processor. With the support of MinJie, we managed to finish the first generation of XiangShan in 10 months, which is taped out at a frequency of 1.3 GHz on 28 nm. We further optimize it toward the second generation in an agile way, which is scheduled to be taped out at 2 GHz on 14 nm in the second quarter (Q2) of 2023. We estimate that the second generation can achieve 9.52/GHz on the SPECint 2006.

We further discuss the potential and long-term impacts of XiangShan and MinJie. They are promising to promote new research directions and drive innovation forward in both academia and industry.

DESIGN AND IMPLEMENTATION OF THE XIANGSHAN PROCESSORS

As a superscalar, out-of-order RISC-V processor, XiangShan is designed in the Chisel hardware construction language (HCL). Since June 2020, XiangShan has been open sourced on major source-code-hosting platforms, with a full development history of the first two microarchitecture generations, Yanqihu (YQH) and Nanhu (NH), and the ongoing third generation, Kunminghu (KMH).

Figure 1 shows the layout of the first and second generations of XiangShan processors and the motherboard for the YQH test chip. As the first generation of XiangShan, Yanqihu (YQH) was taped out on 28 nm in July 2021 with an operating frequency of 1.3 GHz. The chip was successfully brought up in February 2022 and achieves the expected 7.03/GHz SPECint 2006 performance on DDR4-1600. As the second generation, a dual-core NH implementation will be taped out on 14 nm in Q2 2023. It is estimated to reach 9.52/GHz single-threaded SPECint 2006 performance at 2 GHz.

Table 1 highlights the typical configurations of the first two generations of XiangShan. Due to the flexibility of Chisel designs, most parameters can be configured based on PPA requirements.

In the front end, the NH microarchitecture features a decoupled style of a branch prediction unit (BPU) and an instruction fetch unit, where the BPU runs ahead and provides sufficient instruction fetch requests. It adopts

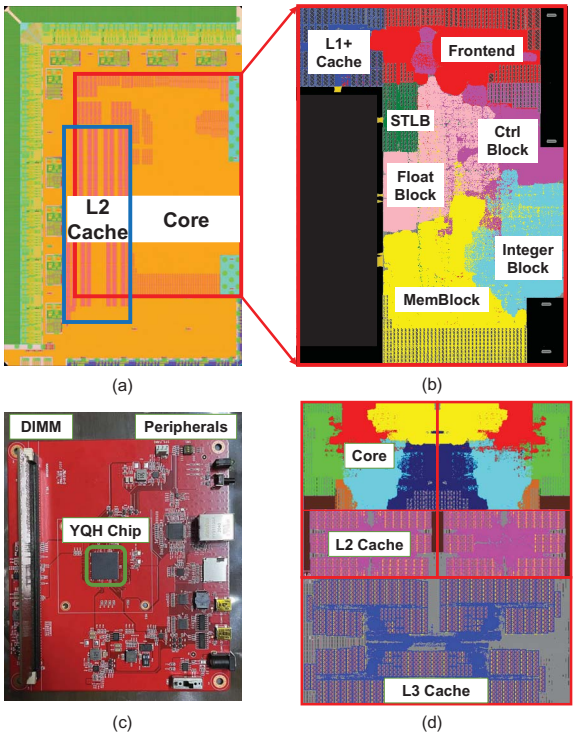


FIGURE 1. Layout and motherboard of XiangShan. (a) Layout of YQH chip. (b) Layout of YQH core. (c) YQH chip and motherboard. (d) Layout of dual-core NH chip. Ctrl: control; L: level; MemBlock: memory block; DIMM: Dual Inline Memory Module; STLB: shared translation lookaside buffer.

a four-table 16,384-entry TAGE-SC branch predictor with an indirect-jump predictor. In the decode stage, macro-operation fusion is utilized such that certain consecutive arithmetic instructions can be fused into a single micro-operation to reduce the execution latency and improve the utilization of pipelines and buffers.

In the execution engine, NH implements separate general-purpose and floating-point register files. Move elimination is enabled by a reference-counting mechanism for the integer physical registers. The reservation stations are distributed and grouped as smaller ones that issue one or two instructions to the execution units in every clock cycle. Cascade floating-point multiply–add (FMA) units, with a total execution latency of five cycles, allow an FMA instruction to be issued early when only the two multiplication operands are ready and to be issued again after the third operand is ready.

In the memory subsystem, three levels of caches, especially including the level-1 (L1) instruction cache, are coherent via the 256-bit TileLink interconnect. With a customized hardware-managed cross-level anti-aliasing algorithm, NH supports larger virtually indexed,

Authorized licensed use limited to: ShanghaiTech University. Downloaded on November 10,2026 at 03:16:58 UTC from IEEE Xplore. Restrictions apply.

TABLE 1. Microarchitectural parameters of the two generations of XiangShan for tape-out: YQH and NH.*

Feature	YQH	NH
ISA	RV64GC	RV64GCBK
Process node	28 nm	14 nm
Core count	1	2
Die size	8.6 mm^2	22.13 mm^2
Standard cell number/area	5.05 million/ 4.27 mm^2	11.3 million/ 4.53 mm^2
Memory number/area	261/1.7 mm^2	692/8.93 mm^2
Density	66%	35%
Frequency	1.3 GHz, TT 0.9 V	2GHz, TT 0.9 V
microBTB	32 entries	256 entries
BTB	2048 entries	4096 entries
TAGE-SC	16,384 entries	16,384 entries
Others	RAS	RAS and ITTAGE
Fetch width	8*4-byte instructions/ cycle	8*4-byte instructions/ cycle
Decode/ Rename width	6 instructions/ cycle	6 instructions/ cycle
ROB/LQ/SQ	192/64/48	256/80/64
Phy. Int/ FP RF	160/160	192/192
L1 instruction TLB	40 entries	40 entries
L1 data TLB	40 entries	136 entries
L2 TLB	4096 entries	2048 entries
L1 instruction cache	16 KB, four-way	64 KB, four-way
L1+ cache	128 KB, eight-way	—
L1 data cache	32 KB, eight-way	64 KB, four-way
L2 cache	1 MB, eight-way, inclusive	1 MB, eight-way, noninclusive
L3 cache	—	6 MB, six-way, noninclusive

*ISA: instruction set architecture; L: level; TT: typical–typical; BTB: branch target buffer; RAS: return address stack; ITTAGE: Indirect Target TAgged GEometric length predictor; ROB: re-order buffer; LQ: load queue; SQ: store queue; Phy. Int/FP RF: physical integer/floating-point register file; TLB: translation lookaside buffer.

physically tagged L1 caches. In addition, the L2 cache is equipped with an effective hybrid prefetching mechanism, combining the best-offset and spatial memory streaming strategies.

MINJIE: AGILE CHIP DEVELOPMENT INFRASTRUCTURES AND PLATFORM

As the development environment for the XiangShan processors, MinJie is an open source platform visioning an end-to-end agile chip development workflow. It has integrated a rich set of open source tools to enhance the design and verification methodologies for high-performance RISC-V processors. Figure 2 shows some of the MinJie toolchains for logic design, functional verification, and performance evaluation.

For register transfer level (RTL) design, though Verilog has long been the mainstream hardware description language, embedded domain-specific languages for hardware construction have become popular in recent years. MinJie provides rich support for the Scala-hosted Chisel, including new hardware programming paradigms in Scala, advanced utility libraries for high-performance designs in Chisel, and customized FIRRTL transforms to automate the end-to-end development workflow.

MinJie offers multiple test generation techniques for functional verification. The AbstractMachine project provides an abstraction layer for freestanding C runtime and enables the development of bare-metal programs with unified application programming interfaces for different architectures, including different execution environments such as instruction set architecture (ISA) emulation, RTL simulation, field-programmable gate array (FPGA) prototyping, and application-specific integrated circuit chips. Generic checkpoints (GCPT) implement the architectural checkpoints for RISC-V and generates arbitrary program slices from real-world workloads. Traditional algorithms, such as constrained random and mutation-based fuzzing, are also supported. More details on the functional verification workflow, such as reference models (REFs), result checking, and debugging, are detailed later.

MinJie also integrates several accurate presilicon performance evaluation tools, including BetaPoint for RTL simulation and Accurate Memory Controller Evaluation (AMCE) for FPGAs. To enhance the accuracy of sampling-based performance estimation, BetaPoint generates personalized microarchitectural warm-up profiles for program slices with different footprints in branch predictors and caches. To address the inconsistent frequency ratio between the CPU (10–100 MHz) and DDR4 (up to 2400 MHz), AMCE designs synthesizable memory

Authorized licensed use limited to: ShanghaiTech University. Downloaded on November 10,2026 at 03:16:58 UTC from IEEE Xplore. Restrictions apply.

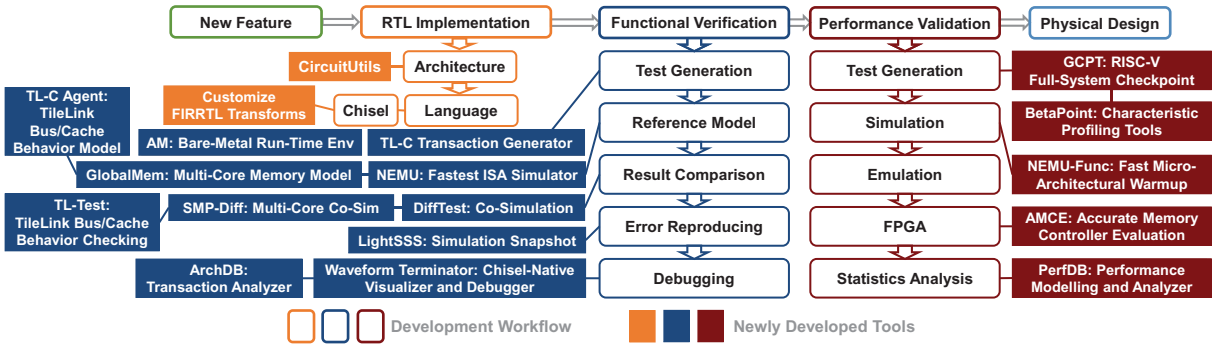


FIGURE 2. Agile workflow of MinJie and the related tools used in different stages of the chip development process. Co-Sim: cosimulation; FPGA: field-programmable gate array; GlobalMem: global memory; ISA: instruction set architecture; LightSSS: lightweight simulation snapshot; TL: TileLink; TL-C: TileLink Cached; FIRRTL: Flexible Internal Representation for RTL; AM: AbstractMachine; Env: environment; SMP-Diff: symmetric multiprocessor differential testing; ArchDB: architecture database; NEMU: New Emulator; GCPT: generic checkpoints.

models and enables precise CPU memory behaviors on FPGAs.

In the following sections, we detail the functionalities and design methods of representative tools for agile verification and debugging.

Diff-Rule-Based Agile Verification (DRAV)

The common practice of functional verification for general-purpose processors is to build a cosimulation framework[4,5] to compare the equivalence between the outcomes of the design under test (DUT) and a REF. For example, to ensure the ISA-level compliance, architectural registers are usually selected as indicators for the program behavior. When the DUT outputs a different register value from that of the REF, a bug report is generated by the cosimulation framework.

In reality, since the instruction set manuals are written in natural languages and not executable, the actual REFs used for verification are instruction set simulators (ISSs), like QEMU, that implement only one or several simplified variants of the ISA model.

However, there will be scenarios where designs are legally allowed by the ISA to have diverse architectural behaviors, such as the timing of handling asynchronous external interrupts. For example, in an in-order five-stage processor, typically, the instruction at the decode stage serves the interrupt. By contrast, XiangShan always raises interrupts along with the oldest interrupt-safe instruction. However, an ISS without detailed pipelines responds to an interrupt immediately on the next unfinished instruction. If we cosimulate the ISS with either the five-staged pipelined processor or XiangShan, a bug will be reported after the interrupt is serviced. This is actually a false positive because the asynchronous behaviors of all three implementations are compatible with the ISA model.

As shown in the example, though RISC-V compatible, different designs still appear to behave nondeterministically, even with the same stimuli. The diverse behaviors are caused mainly by microarchitectural implementation details and probably result in false positives during cosimulation.

Therefore, in consideration of the diversity an ISA would allow, any cosimulation mechanism for processor functional verification must address the issue of *behavioral nondeterminism*. This problem is a widely accepted challenge and drives important research directions in hardware formal verification,[6] software testing,[7] and software security.[8]

Traditionally, processors are designed and verified for a long time. The problem can be tackled by setting up detailed golden REFs specifically targeted at certain design parts to thoroughly eliminate any nondeterministic possibilities. This fine-grained and tedious verification strategy ensures a *one-to-one correspondence* between the DUT and the REF. This model is useful for designs, such as commercial processors, with a long production lifecycle where the verification intellectual properties and frameworks can be reused throughout the project.

Nevertheless, maintaining the one-to-one correspondence between the DUT and REF can be difficult with a short development cycle, rapid feature changes, and limited resources, which are commonly seen in the agile development model. If we rethink the principle of verification, a given design specification can lead to

Authorized licensed use limited to: ShanghaiTech University. Downloaded on November 10,2026 at 03:16:58 UTC from IEEE Xplore. Restrictions apply.

diverse implementations. Therefore, once the behavior of a DUT satisfies the definition of the specifications, the implementation details are allowed to be diverse, and the requirement of strict equivalence between the architectural registers of the DUT and REF can be relaxed.

Following this principle, it is feasible that DUTs with different implementation details can be verified with the same REF if their behaviors conform to the same specification. This forms an N-*to-one mapping* between DUTs and the REF. Since only one simpler REF is maintained, the functional verification overhead for multiple processors is reduced.

Based on the observations, we propose a novel *DRAV* mechanism. We leverage *diff-rules* to abstract the legal behaviors defined in specifications and deal with nondeterminism in verification. A diff-rule is deterministic and stable across different implementations with the same design objectives. DRAV enhances conventional verification by providing both the methodology and an implementation for RISC-V processors.

DiffTest: DRAV for RISC-V Processors

DiffTest is a cosimulation-based verification framework for RISC-V processors that accelerates functional verification by adopting the DRAV methodology. It provides the flexibility to add diff-rules and reconfigure the reference model on the fly, thus being scalable to dynamically support multiple designs.

The key challenge of devising diff-rules is identifying sources of nondeterministic microarchitecture-dependent behaviors in the RISC-V architecture. In this section, we introduce in detail two representative sources of nondeterminism in multicore, high-performance RISC-V processors.

Speculative Virtual Address Translations

In the RISC-V Linux kernel, the operating system chooses not to execute a memory-barrier instruction after allocating a new physical page to avoid flushing instructions until a page fault exception. In most cases, the in-memory page table entries (PTEs) are updated quickly after the retirement of the store instructions. However, if the store operation that updates a PTE does not take effect when the translation look-aside buffer accesses the memory, possibly due to the existence of a store queue or a store buffer, a memory instruction accessing the page will trigger a page fault exception. Two diff-rules are involved in addressing this issue: 1) the DUT may trigger a page fault exception even if the REF does not trigger, and 2) the DUT and REF should have the same architectural states after executing the same instruction.

Cache Hierarchy and Multicore Scenarios

Load and store instructions may have different outcomes due to the exponential interleaving space of concurrent memory accesses. We leverage diff-rules to prune the astronomically large interleaving space and build a cosimulation framework for multicore processors using simple single-core REFs.

As an example, under the RISC-V weak memory order (RVWMO) model, load instructions are allowed to first bypass the value from the private store buffer and access the global memory if the bypass fails. A naive multicore cosimulation requires maintaining a correct store buffer in the REFs unless disagreements between the DUT and REF will frequently break the cosimulation flow. In DiffTest, we devise a diff-rule from the RVWMO specification that allows DUTs to maintain the global memory and updates the REF memory when they disagree. It introduces GlobalMemory (GM) that records the store requests that enter the cache hierarchy in the DUT. Data correctness is checked by both GM and the local memory of single-core REFs. When the single-core REF executes a load with a different value from the DUT, DiffTest accesses the same load address in GM to check whether other hardware threads possibly write this load value. If so, the value will be updated to both the local memory and the destination register of the load instruction in the single-core REF.

Lightweight Simulation Snapshot (LightSSS): Efficient On-Demand Debugging

Debugging requires information, such as the waveform and logs, that slows down the simulation speed. We propose LightSSS, a technique for on-demand transitions between fast mode and debug mode (debugging information enabled).

Figure 3 demonstrates various ways to obtain debugging information. As shown in Figure 3(a), due to the extra debugging information acquired, RTL simulation in debug mode is usually much slower than in normal mode, regardless of the format of the information. An example of XiangShan shows that, when the waveform is enabled, the RTL simulation speed drops to $\sim$8.5%, whereas the export of text logs causes an $\sim$55% decrease of simulation speed as well.

As shown in Figure 3(b), a straightforward method[9] trades off simulation resources for quicker reproduction. It runs two identical RTL simulation instances in parallel, with one instance (`S0`) launched N seconds

Authorized licensed use limited to: ShanghaiTech University. Downloaded on November 10,2026 at 03:16:58 UTC from IEEE Xplore. Restrictions apply.

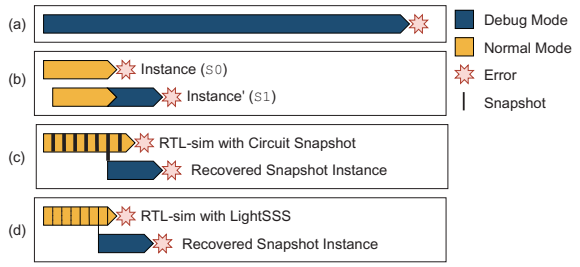


FIGURE 3. Various ways to obtain debugging information. (a) Always in debug mode. (b) Running two RTL simulation instances successively. (c) RTL simulation with existing circuit-level snapshots. (d) RTL simulation with LightSSS. sim: simulation.

later than the other (S1). When S0 reports a bug, it notifies S1 to turn on the debugging information, like waveform, and continue the simulation until the bug occurs. Since S1 is identical to S0 except for the delay of N seconds, when S1 reports the same bug, we can obtain the debugging information for the last N seconds [region of interest (ROI)] of the simulation. This method avoids simulating in debug mode most times but requires doubled computation resources.

Another debugging strategy shown in Figure 3(c) is to periodically take snapshots of the simulated hardware circuit and restore a recent snapshot on demand. This method allows for a fast switch across all simulation timestamps for debugging but suffers from a significant performance overhead of creating and storing the snapshots. For example, the state-of-the-art RTL simulation snapshot tool LiveSim[10] causes a 10%–20% performance degradation.

To enable an agile debugging workflow, MinJie integrates the LightSSS technique into DiffTest. As illustrated in Figure 3(d), LightSSS considerably reduces the overhead of snapshots and still allows debugging of the ROI with far less resource waste. First, it preserves only the two most recent snapshots in memory and drops any outdated ones. Second, instead of creating complete snapshots, it records in memory only the parts that differ between snapshots and the current status. Furthermore, it generates process-level snapshots that are independent of the simulated circuit structure.

LightSSS is implemented using the highly efficient `fork()` system call of the Linux kernel. During RTL simulation, LightSSS periodically calls `fork()` and treats the forked process as a snapshot. The forked process waits for the parent process to exit abnormally, possibly because DiffTest detects a functional bug. Meanwhile, as the parent process continues and updates the memory, the copy-on-write forking strategy inherently creates incremental snapshots: the operating system allocates physical pages for only modified pages, leaving unmodified pages shared between processes. Since LightSSS allows the existence of, at most, two snapshots, older snapshots are killed and do not consume any resources. Evaluation results show that LightSSS causes minor performance overhead in typical scenarios—less than 1% of the RTL simulation speed and an order of magnitude less than LiveSim.

POTENTIAL: LONG-TERM IMPACT

> "The theory behind open source is simple. In the case of an operating system, the source code is free. Anyone can improve it, change it, exploit it. But those improvements, changes, and exploitations have to be made freely available"—Linus Torvalds.[11]

Over the past decade, agile and open source hardware has gained increasing attention in both academia and industry. Inspired by the open source software ecosystem, the open source hardware ecosystem can be classified into three levels:

- *L1*: Instruction sets are open and free. Since instruction sets are essentially specifications, L1 indicates that instruction sets should be freely available.
- *L2*: The design and implementation are open and free. A hardware project belonging to L2 means that the source code and related documentation should be open sourced and can be freely accessed.
- *L3*: The development infrastructure is open and free. To develop a chip, one may require a set of tools, including electronic design automation tools for hardware design, implementation, verification, and so on. L3 indicates that these tools should be open and free.

RISC-V is widely recognized as a member of L1 and has made a significant impact worldwide. Among the RISC-V implementations, at least 30 of them, as listed in the RISC-V manual, are publicly accessible and have been allocated canonical open source architecture IDs. However, due to the complex and multistage nature of the chip development process, the hardware industry highly relies on commercial licenses and toolchains from third-party vendors and in-house services. As a result, the substantial costs associated with an agile and open

Authorized licensed use limited to: ShanghaiTech University. Downloaded on November 10,2026 at 03:16:58 UTC from IEEE Xplore. Restrictions apply.

source hardware project require a significant, long-term investment that is orders of magnitude higher than for software development. Such an investment is not feasible for a single company or research institute to undertake. Consequently, there are still few successful open source hardware projects belonging to L2 and L3.

AMONG THE RISC-V IMPLEMENTATIONS, AT LEAST 30 OF THEM, AS LISTED IN THE RISC-V MANUAL, ARE PUBLICLY ACCESSIBLE AND HAVE BEEN ALLOCATED CANONICAL OPEN SOURCE ARCHITECTURE IDs.

In this article, we present XiangShan and MinJie, both of which are open source and freely available, which belong to L2 and L3, respectively.

To establish a continuously advancing, competitive, and open hardware ecosystem, the nonprofit organization Beijing Institute of Open Source Chip (BOSC) was founded with industry-leading companies as its founding members. Working closely with the industry, BOSC will further develop XiangShan and MinJie.

Over the past three years, XiangShan has emerged as one of the most active open source hardware projects, garnering more than 3500 stars and 400 forks on source-code-hosting platforms. As far as we know, XiangShan has also become the fastest open source RISC-V processor, and its third generation, KMH, is poised to deliver even higher performance.

Looking ahead, we firmly believe that XiangShan and MinJie will demonstrate the viability and potential of open source hardware projects and continue to exert significant influence in the computer architecture community, akin to the remarkable impact Linux has made as an exceptional research platform that bridges the gap between academia and industry.

DRAV

As customized architectures emerge and design complexity increases, the efficient construction of REFs will become a critical issue and may be a bottleneck for functional verification. This article presents a novel DRAV mechanism to address the nondeterminism in design specifications and reduce the overhead of REFs with diff-rules. DRAV provides a uniform abstraction for specifying the nondeterministic yet legal behaviors of the DUT. In particular, it provides a formalized description of the nondeterminism problem that can be integrated with existing methods to verify hardware designs.

Computer Architecture Research Platform

As the fastest open source processor, XiangShan has raised the performance ceiling of publicly accessible processors and established the competitive groundwork for future computer architecture research. On the one hand, the development of XiangShan processors presents significant challenges for microarchitecture design and encourages innovation. On the other hand, XiangShan, in conjunction with MinJie, has the potential to become one of the most crucial infrastructures for computer architecture researchers, enabling them to test novel ideas on an open and collaborative platform. As demonstrated in our MICRO'22 paper,[12] a third-year Ph.D. student was able to efficiently implement a microarchitecture technique on XiangShan within a few hours and validate its effectiveness using MinJie tools. Additionally, with industry backing, there is a routine tape-out plan for identifying and certifying silicon-proven research

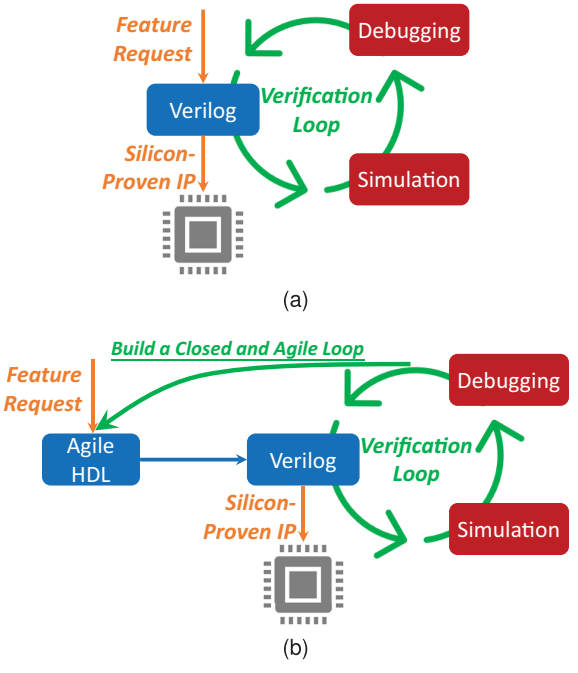


FIGURE 4. MinJie envisions an agile development workflow. (a) Traditional workflow. (b) Agile workflow. HDL: hardware description language.

Authorized licensed use limited to: ShanghaiTech University. Downloaded on November 10,2026 at 03:16:58 UTC from IEEE Xplore. Restrictions apply.

ideas. This continued support will be a robust catalyst, further accelerating the collaboration between academia and industry.

Agile Hardware Development Platform

As an open platform, MinJie has integrated a series of development toolchains and will continue to incorporate future architecture research tools, such as power modeling, formal verification, and secure enclave tools. Together with XiangShan, it has the potential to become an active platform for verifying novel methodologies on real-world, large-scale hardware designs.

TOGETHER WITH XIANGSHAN, MINJIE HAS THE POTENTIAL TO BECOME AN ACTIVE PLATFORM FOR VERIFYING NOVEL METHODOLOGIES ON REAL-WORLD, LARGE-SCALE HARDWARE DESIGNS.

Workflow integration, as one of the most critical issues we have identified in agile chip development, is illustrated in Figure 4. As shown in Figure 4(a), conventional hardware development is mostly centered on Verilog. With Verilog-based designs, the traditional verification flow is well supported by a plethora of commercial tools. However, the agile hardware design paradigm favors high-level HCLs with higher coding efficiency. As existing verification tools are primarily optimized for Verilog designs, the adoption of high-level HCLs results in a broken workflow between the design and verification stages. In response to this challenge, MinJie proposes some novel tools, such as information probes and ArchDB, to bridge the gap between agile hardware design and the verification paradigm. In our MICRO'22 paper,[12] we demonstrate the feasibility and effectiveness of a closed and agile loop. As shown in Figure 4(b), these tools automatically extract information from high-level designs and bridge the gap between the agile hardware design and verification stages.

CONCLUSION

We introduce MinJie, an agile development platform for high-performance chip designs that incorporates a broad set of tools for agile development workflow, and XiangShan, an industry-competitive, high-performance RISC-V processor. We have also discussed the promising future directions of XiangShan and MinJie as an exceptional research platform for both academia and industry. With the open source nature of both XiangShan and MinJie, we believe that they will continue to push the boundaries of computer architecture research and development.

ACKNOWLEDGMENTS

We give special thanks to our team members and collaborators. This work was carried by the following coauthors: Guokai Chen, Lu Chen, Lingrui Gou, Yue Jin, Qianruo Li, Xin Li, Zuojun Li, Jiawei Lin, Tong Liu, Zhigang Liu, Jiazhan Tan, Huaqiang Wang, Huizhe Wang, Kaifan Wang, Chuanqi Zhang, Fawang Zhang, Linjuan Zhang, Zifei Zhang, Yangyang Zhao, Yaoyang Zhou, Yike Zhou, Jiangrui Zou, Zusong Li, Jiye Zhao, Zihao Chen, Qiyuan Quan, Xingwu Liu, Sa Wang, and Kan Shi. This work was supported in part by the Strategic Priority Research Program of Chinese Academy of Sciences (Grant XDC05030200), National Key R&D Program of China (Grant 2022YFB4500403), National Natural Science Foundation of China (Grant 62090022, Grant 62172388, and Grant 62072433), Youth Innovation Promotion Association of the Chinese Academy of Sciences (Grant 2020105), Institute of Computing Technology (ICT) Innovation Grant E261100, and Fundamental Research Funds for the Central Universities (Grant DUT21RC(3)102 and Grant DUT21LAB302).

REFERENCES

1. Y. Lee et al., "An agile approach to building RISC-V microprocessors," *IEEE Micro*, vol. 36, no. 2, pp. 8–20, Mar./Apr. 2016, doi: 10.1109/MM.2016.11.
2. Y. Bao and T. E. Carlson, "Agile and open-source hardware," *IEEE Micro*, vol. 40, no. 4, pp. 6–9, Jul./Aug. 2020, doi: 10.1109/MM.2020.3002606.
3. C. Yuan, X. Gao, Y. Chen, and Y. Bao, "Teaching undergraduates to build real computer systems," *Commun. ACM*, vol. 64, no. 11, pp. 48–49, Oct. 2021, doi: 10.1145/3481606.
4. N. Kabylkas, T. Thorn, S. Srinath, P. Xekalakis, and J. Renau, "Effective processor verification with logic fuzzer enhanced co-simulation," in *Proc. 54th Annu. IEEE/ACM Int. Symp. Microarchit.*, 2021, pp. 667–678, doi: 10.1145/3466752.3480092.
5. K. Shi, S. Xu, Y. Diao, D. Boland, and Y. Bao, "ENCORE: Efficient architecture verification framework with FPGA acceleration," in *Proc. ACM/SIGDA Int. Symp. Field Programmable Gate Arrays*, 2023, pp. 209–219, doi: 10.1145/3543622.3573187.
6. A. Fedotov and J. Schmaltz, "Automatic generation of hardware checkers from formal micro-architectural specifications," in *Proc. Des., Automat Test Europe*

Authorized licensed use limited to: ShanghaiTech University. Downloaded on November 10,2026 at 03:16:58 UTC from IEEE Xplore. Restrictions apply.

Conf. Exhib., 2018, pp. 1568–1573, doi: 10.23919/DATE.2018.8342265.
7. C. Liu, S. Gong, and P. Fonseca, "KIT: Testing OS-level virtualization for functional interference bugs," in *Proc. 28th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, 2023, vol. 2, pp. 427–441, doi: 10.1145/3575693.3575731.
8. S. Qin, C. Zhang, K. Chen, and Z. Li, "iDEV: Exploring and exploiting semantic deviations in ARM instruction processing," in *Proc. 30th ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2021, pp. 580–592, doi: 10.1145/3460319.3464842.
9. D. Kim, C. Celio, S. Karandikar, D. Biancolin, J. Bachrach, and K. Asanović, "DESSERT: Debugging RTL effectively with state snapshotting for error replays across trillions of cycles," in *Proc. 28th Int. Conf. Field Program. Logic Appl.*, 2018, pp. 76–764, doi: 10.1109/FPL.2018.00021.
10. H. Skinner, R. Trapani Possignolo, S.-H. Wang, and J. Renau, "LiveSim: A fast hot reload simulator for HDLs," in *Proc. IEEE Int. Symp. Perf. Anal. Syst. Softw.*, 2020, pp. 126–135, doi: 10.1109/ISPASS48437.2020.00028.
11. L. Torvalds and D. Diamond, *Just for Fun: The Story of an Accidental Revolutionary*. New York, NY, USA: Harper Collins, 2001.
12. Y. Xu et al., "Towards developing high performance RISC-V processors using agile methodology," in *Proc. 55th IEEE/ACM Int. Symp. Microarchit.*, 2022, pp. 1178–1199, doi: 10.1109/MICRO56248.2022.00080.

YINAN XU is a Ph.D. candidate at the State Key Lab of Processors, Institute of Computing Technology (ICT), Chinese Academy of Sciences (CAS), and the University of Chinese Academy of Sciences, Beijing, 100190, China. His research interests include the agile development of processors and computer architecture. Xu received a B.Eng. degree in computer science and technology from the University of Chinese Academy of Sciences (UCAS), Beijing, China. He is a Graduate Student Member of IEEE. Contact him at xuyinan@ict.ac.cn.

ZIHAO YU is an engineer at the State Key Lab of Processors, ICT, CAS, Beijing, 100190, China. His research interests include computer architectures and operating systems. Yu received a Ph.D. degree in computer systems organization from ICT, CAS. Contact him at yuzihao@ict.ac.cn.

DAN TANG is the assistant director at the Beijing Institute of Open Source Chip (BOSC), Beijing, 100080, China and a senior engineer at the State Key Lab of Processors, ICT, CAS. His main research interests include computer architecture and low power system-on-chip design. Tang received a Ph.D. degree in computer systems organization from ICT, CAS. Contact him at tangdan@bosc.ac.cn.

YE CAI is an associate professor at Shenzhen University, Shenzhen, 518060, China. His main research interests include computer architecture and computer system. Cai received a Ph.D. degree in computer systems organization from ICT, CAS. Contact him at caiye@szu.edu.cn.

DANDAN HUAN is an engineer at Beijing VCore Technology, Beijing, 100190, China. Her main research interests include high-performance computer architecture and microprocessor design. Huan received a Ph.D. degree in computer systems organization from ICT, CAS. Contact her at huandandan@rvcore.com.

WEI HE is an engineer at Peng Cheng Laboratory, Shenzhen, 518000, China. His main research interests include high-speed digital circuit design and low power design. He received a M.Eng. degree in communication and information systems from Xidian University. Contact him at hew@pcl.ac.cn.

NINGHUI SUN is a professor at the State Key Lab of Processors, ICT, CAS, and an academician of the Chinese Academy of Engineering, Beijing, 100190, China. His research interests include computer architecture and high-performance computing. Sun received a Ph.D. degree in computer systems organization from ICT, CAS. Contact him at snh@ict.ac.cn.

YUNGANG BAO is a professor at the State Key Lab of Processors, ICT, CAS; the deputy director of the ICT; and the chief scientist at BOSC, Beijing, 100080, China. His research interests include open source hardware and agile chip design, data center architecture, and memory systems. Bao received a Ph.D. degree in computer systems organization from ICT, CAS. Contact him at baoyg@ict.ac.cn.

Authorized licensed use limited to: ShanghaiTech University. Downloaded on November 10,2026 at 03:16:58 UTC from IEEE Xplore. Restrictions apply.